МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
“БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ”
ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №6

Специальность ПО-13

Выполнил:
М.А.Шумило
студент группы
ПО-13
Проверил:
А.Д.Кулик
25.03.2026

Брест 2026

Цель работы: освоить приемы тестирования кода на примере использования пакета pytest

Ход работы

Задание 1: Написание тестов для мини-библиотеки покупок (shopping.py)
1. Создайте файл test_cart.py. Реализуйте следующие тесты:
   - Проверка добавления товара: после add_item("Apple", 10.0) в корзине должен быть один элемент.
   - Проверка выброса ошибки при отрицательной цене.
   - Проверка вычисления общей стоимости (total()).
2. Протестируйте метод apply_discount с разными значениями скидки:
   - 0% - цена остаётся прежней
   - 50% - цена уменьшается вдвое
   - 100% - цена становится ноль
   - < 0% и > 100% - должно выбрасываться исключение

Используйте @pytest.mark.parametrize
3. Создайте фикстуру empty_cart, которая возвращает пустой экземпляр Cart
@pytest.fixture
def empty_cart():
return Cart()
Используйте эту фикстуру в тестах, где нужно создать новую корзину.
4. Допустим, у нас есть функция, которая логирует покупку в удалённую систему:
import requests
def log_purchase(item):
requests.post("https://example.com/log", json=item)
   - Замокайте requests.post, чтобы не было реального HTTP-запроса
   - Убедитесь, что он вызывается с корректными данными

5. Добавьте поддержку купонов:
def apply_coupon(cart, coupon_code):
coupons = {"SAVE10": 10, "HALF": 50}
if coupon_code in coupons:
cart.apply_discount(coupons[coupon_code])
else:
raise ValueError("Invalid coupon")
   - Напишите тесты на apply_coupon
   - Замокайте словарь coupons с помощью monkeypatch или patch.dict

Выполнение:

Код программы:
shopping.py:
```
import requests

class Cart:
    def __init__(self):
        self.items = []

    def add_item(self, name, price):
        if price < 0:
            raise ValueError("Price cannot be negative")
```

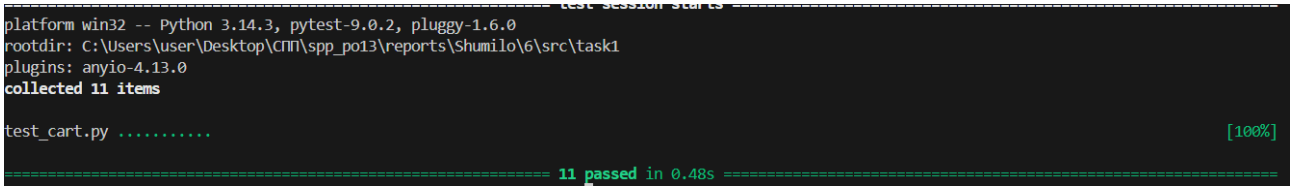
```
        self.items.append({"name": name, "price": price})

    def total(self):
        return sum(item["price"] for item in self.items)

    def apply_discount(self, percent):
        if percent < 0 or percent > 100:
            raise ValueError("Invalid discount percent")

        multiplier = (100 - percent) / 100
        for item in self.items:
            item["price"] *= multiplier

def log_purchase(item):
    requests.post("https://example.com/log", json=item)

coupons = {"SAVE10": 10, "HALF": 50}

def apply_coupon(cart, coupon_code):
    if coupon_code in coupons:
        cart.apply_discount(coupons[coupon_code])
    else:
        raise ValueError("Invalid coupon")
```

test_cart.py:

```
import pytest
from unittest.mock import patch
from shopping import Cart, log_purchase, apply_coupon

# Фикстура
@pytest.fixture
def empty_cart():
    return Cart()


# Тесты корзины
def test_add_item(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    assert len(empty_cart.items) == 1

def test_add_item_negative_price(empty_cart):
    with pytest.raises(ValueError):
        empty_cart.add_item("Apple", -5.0)

def test_total(empty_cart):
    empty_cart.add_item("Apple", 10.0)
    empty_cart.add_item("Banana", 5.0)
    assert empty_cart.total() == 15.0

# Тесты apply_discount
@pytest.mark.parametrize(
    "discount, expected_total",
    [
        (0, 100.0),
        (50, 50.0),
        (100, 0.0),
    ]
)
def test_apply_discount_valid(empty_cart, discount, expected_total):
    empty_cart.add_item("Item", 100.0)
    empty_cart.apply_discount(discount)
    assert empty_cart.total() == expected_total

@pytest.mark.parametrize("discount", [-10, 150])
```

```
def test_apply_discount_invalid(empty_cart, discount):
    empty_cart.add_item("Item", 100.0)
    with pytest.raises(ValueError):
        empty_cart.apply_discount(discount)

# Тесты log_purchase
@patch("shopping.requests.post")
def test_log_purchase(mock_post):
    item = {"name": "Apple", "price": 10.0}

    log_purchase(item)

    mock_post.assert_called_once_with(
        "https://example.com/log",
        json=item
    )

# Тесты apply_coupon
def test_apply_coupon_valid(empty_cart, monkeypatch):
    empty_cart.add_item("Item", 100.0)

    fake_coupons = {"SAVE10": 10}
    monkeypatch.setattr("shopping.coupons", fake_coupons, raising=False)

    apply_coupon(empty_cart, "SAVE10")

    assert empty_cart.total() == 90.0

def test_apply_coupon_invalid(empty_cart, monkeypatch):
    empty_cart.add_item("Item", 100.0)

    fake_coupons = {"SAVE10": 10}
    monkeypatch.setattr("shopping.coupons", fake_coupons, raising=False)
    with pytest.raises(ValueError):
        apply_coupon(empty_cart, "BADCODE")
```

Рисунки с результатами работы программы



Задание 2

Напишите тесты к реализованным функциям из лабораторной работы No1. Проверьте тривиальные и граничные случае, а также варианты, когда может возникнуть исключительная ситуация. Если при реализации не использовались отдельные функции, необходимо провести рефакторинг кода.

Выполнение:

Код программы:

lab1.py:

```
def parse_int_list(raw: str) -> list[int]:
    """Преобразует строку в список целых чисел.
       Бросает ValueError, если хотя бы один элемент не int."""
    nums = []
    for item in raw.split():
        try:
```

```
            nums.append(int(item))
        except ValueError:
            raise ValueError(f"'{item}' не является целым числом")
    return nums

def digit_distribution(nums: list[int]) -> dict[int, int]:
    """Возвращает распределение чисел по количеству цифр."""
    dist = {}
    for n in nums:
        digits = len(str(abs(n)))
        dist[digits] = dist.get(digits, 0) + 1
    return dist

def hamming_weight(x: int) -> int:
    """Количество установленных битов."""
    if not isinstance(x, int):
        raise TypeError("Аргумент должен быть int")
    if x < 0:
        raise ValueError("Число должно быть неотрицательным")
    return bin(x).count("1")
```

test_lab1.py:

```
import pytest
from lab1 import parse_int_list, digit_distribution, hamming_weight


# ТЕСТЫ ДЛЯ parse_int_list
def test_parse_int_list_valid():
    """
    Проверяем корректный случай:
    строка содержит только целые числа.
    """
    assert parse_int_list("1 2 3") == [1, 2, 3]

def test_parse_int_list_negative():
    """
    Проверяем, что отрицательные числа тоже корректно парсятся.
    """
    assert parse_int_list("-1 -2 -3") == [-1, -2, -3]

def test_parse_int_list_invalid():
    """
    Проверяем, что при наличии нечислового элемента выбрасывается ValueError.
    """
    with pytest.raises(ValueError):
        parse_int_list("10 abc 20")

def test_parse_int_list_empty_string():
    """
    Граничный случай: пустая строка → пустой список.
    """
    assert parse_int_list("") == []

# ТЕСТЫ ДЛЯ digit_distribution
def test_digit_distribution_simple():
    """
    Проверяем базовый случай:
    числа разной длины → корректное распределение.
    """
    nums = [1, 22, 333]
    assert digit_distribution(nums) == {1: 1, 2: 1, 3: 1}
```

```python
def test_digit_distribution_zero():
    """
    Число 0 имеет одну цифру.
    """
    assert digit_distribution([0]) == {1: 1}

def test_digit_distribution_repeated():
    """
    Несколько чисел одинаковой длины.
    """
    nums = [5, 7, 12, 99]
    assert digit_distribution(nums) == {1: 2, 2: 2}

def test_digit_distribution_negative_numbers():
    """
    Отрицательные числа должны учитываться по количеству цифр без знака.
    """
    nums = [-1, -22, 333]
    assert digit_distribution(nums) == {1: 1, 2: 1, 3: 1}

def test_digit_distribution_empty():
    """
    Граничный случай: пустой список → пустой словарь.
    """
    assert digit_distribution([]) == {}

# ТЕСТЫ ДЛЯ hamming_weight
def test_hamming_weight_zero():
    """
    0 в двоичном виде — 0, установленных битов нет.
    """
    assert hamming_weight(0) == 0

def test_hamming_weight_normal():
    """
    Проверяем обычный случай:
    5 = 0b101 → два установленных бита.
    """
    assert hamming_weight(5) == 2

def test_hamming_weight_large():
    """
    Граничный случай: число вида 2^n - 1,
    у которого все n младших битов равны 1.
    """
    assert hamming_weight(2**10 - 1) == 10

def test_hamming_weight_negative():
    """
    Функция должна выбрасывать ValueError для отрицательных чисел.
    """
    with pytest.raises(ValueError):
        hamming_weight(-1)

def test_hamming_weight_not_int():
    """
    Функция должна выбрасывать TypeError, если аргумент не int.
    """
    with pytest.raises(TypeError):
        hamming_weight(3.14)
```

Рисунки с результатами работы программы

```
========================================================== test session starts ==========================================================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\user\Desktop\СПП\spp_po13\reports\Shumilo\6\src\task2
plugins: anyio-4.13.0
collected 14 items

test_lab1.py ..............                                                                                                    [100%]

========================================================== 14 passed in 0.07s ==========================================================
```

Задание 3

Написать тесты к методу, а затем реализовать сам метод по заданной спецификации.

5) Реализуйте и протестируйте метод int indexOfDifference(String str1, String str2), который сравнивает две строки и возвращает индекс той позиции, в которой они различаются. Например, indexOfDifference("i am a machine" , "i am a robot") должно вернуть 7.

Спецификация метода:

indexOfDifference (None , None) = TypeError
indexOfDifference ("", "") = -1
indexOfDifference ("", "abc ") = 0
indexOfDifference ("abc ", "") = 0
indexOfDifference ("abc ", " abc ") = -1
indexOfDifference ("ab", " abxyz ") = 2
indexOfDifference (" abcde ", " abxyz ") = 2
indexOfDifference (" abcde ", "xyz") = 0

Выполнение:

Код программы:

task3.py:

```
def indexOfDifference(str1: str, str2: str) -> int:
    # оба None - ошибка
    if str1 is None and str2 is None:
        raise TypeError("Both arguments cannot be None")

    # Если хотя бы один None - тоже ошибка
    if str1 is None or str2 is None:
        raise TypeError("Arguments must be strings")

    # Оба пустые - нет различий
    if str1 == "" and str2 == "":
        return -1

    # Если одна строка пустая, а другая нет - различие в позиции 0
    if str1 == "" or str2 == "":
        return 0

    # Ищем первую позицию, где символы различаются
    min_len = min(len(str1), len(str2))
    for i in range(min_len):
        if str1[i] != str2[i]:
            return i

    # Если все совпадают, но длины разные - различие в конце короткой строки
    if len(str1) != len(str2):
        return min_len
    # Полное совпадение
    return -1
```

test_index_of_difference.py:

```
import pytest
from task3 import indexOfDifference

def test_both_none():
    """indexOfDifference(None, None) = TypeError"""
    with pytest.raises(TypeError):
        indexOfDifference(None, None)

def test_both_empty():
    """indexOfDifference("", "") = -1"""
    assert indexOfDifference("", "") == -1

def test_empty_and_nonempty():
    """indexOfDifference("", "abc") = 0"""
    assert indexOfDifference("", "abc") == 0

def test_nonempty_and_empty():
    """indexOfDifference("abc", "") = 0"""
    assert indexOfDifference("abc", "") == 0

def test_equal_strings():
    """indexOfDifference("abc", "abc") = -1"""
    assert indexOfDifference("abc", "abc") == -1

def test_difference_in_middle():
    """indexOfDifference("i am a machine", "i am a robot") = 7"""
    assert indexOfDifference("i am a machine", "i am a robot") == 7

def test_difference_after_prefix():
    """indexOfDifference("ab", "abxyz") = 2"""
    assert indexOfDifference("ab", "abxyz") == 2

def test_difference_after_prefix2():
    """indexOfDifference("abcde", "abxyz") = 2"""
    assert indexOfDifference("abcde", "abxyz") == 2

def test_difference_at_start():
    """indexOfDifference("abcde", "xyz") = 0"""
    assert indexOfDifference("abcde", "xyz") == 0

def test_one_char_diff():
    assert indexOfDifference("a", "b") == 0

def test_prefix_same_length():
    assert indexOfDifference("abcd", "abce") == 3

def test_longer_second():
    assert indexOfDifference("abc", "abcd") == 3

def test_longer_first():
    assert indexOfDifference("abcd", "abc") == 3
```

Рисунки с результатами работы программы

```
========================================================== test session starts ==========================================================
platform win32 -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\user\Desktop\СПП\spp_po13\reports\Shumilo\6\src\task3
plugins: anyio-4.13.0
collected 13 items

test_index_of_difference.py .............                                                                                    [100%]

========================================================== 13 passed in 0.08s ==========================================================
```

Вывод: освоил приемы тестирования кода на примере использования пакета pytest.